TASKFLOW

Docker & Deploy

Do ambiente local ao ar com Railway ou Render

Versão 1.0 | Abril 2026 | Nível: Iniciante

1. O que é Docker e por que usar

Docker é uma ferramenta que empacota sua aplicação junto com tudo que ela precisa para rodar: sistema operacional base, versão do Python, dependências e código. Esse pacote se chama container.

O resultado é que qualquer pessoa, em qualquer máquina ou servidor, roda exatamente o mesmo ambiente com um único comando — eliminando o clássico problema de 'na minha máquina funciona'.

1.1 Conceitos fundamentais

Conceito	O que é
Imagem	A receita do container. Define sistema operacional, dependências e código. Criada pelo Dockerfile.
Container	A imagem em execução. É o processo real rodando na sua máquina ou servidor.
Dockerfile	Arquivo com instruções para construir a imagem da sua aplicação passo a passo.
Docker Compose	Ferramenta que orquestra múltiplos containers (API + banco) com um único comando.
Volume	Pasta compartilhada entre o container e o disco real. Garante que os dados do banco não se percam.
Network	Rede interna criada pelo Compose para os containers se comunicarem entre si.
Registry	Repositório de imagens. O Docker Hub é o principal — guarda imagens públicas e privadas.

1.2 Como o Docker se encaixa no TaskFlow

Sem Docker você precisaria instalar e configurar Python, PostgreSQL e todas as dependências manualmente. Com Docker, dois containers cuidam de tudo:

- Container api — sua aplicação FastAPI rodando com uvicorn
- Container db — PostgreSQL pronto para uso, sem instalação
- Os dois se comunicam numa rede interna criada automaticamente pelo Compose
- Um volume garante que os dados do banco não se percam ao reiniciar

2. Arquivos de Configuração

2.1 Dockerfile — receita da API

Crie este arquivo na raiz do projeto (mesmo nível do main.py):

```
FROM python:3.11-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

# Roda migrações e sobe o servidor
CMD ["sh", "-c", "alembic upgrade head && uvicorn app.main:app --host 0.0.0.0
--port 8000"]
```

Explicação linha a linha:

- FROM python:3.11-slim — parte de uma imagem oficial do Python (versão enxuta)
- WORKDIR /app — define /app como pasta de trabalho dentro do container
- COPY requirements.txt . — copia só o arquivo de dependências primeiro (otimiza cache)
- RUN pip install — instala as dependências dentro do container
- COPY . . — copia todo o código do projeto
- CMD — comando executado quando o container sobe: migrations + servidor

2.2 docker-compose.yml — orquestra tudo

Crie este arquivo também na raiz do projeto:

```
services:
  api:
    build: .
    ports:
      - "8000:8000"
    env_file:
      - .env
    depends_on:
      db:
        condition: service_healthy
    restart: unless-stopped

  db:
    image: postgres:15
    env_file:
      - .env
    volumes:
```

```
      - pg_data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres"]
      interval: 5s
      timeout: 5s
      retries: 5

volumes:
  pg_data:
```

Explicação dos pontos-chave:

- build: . — usa o Dockerfile da pasta atual para construir a imagem da API
- ports: 8000:8000 — expõe a porta 8000 do container para o seu computador
- env_file: .env — carrega as variáveis de ambiente do arquivo .env
- depends_on + healthcheck — API só sobe quando o banco estiver realmente pronto
- volumes: pg_data — salva os dados do banco no disco mesmo após reiniciar

2.3 .env — variáveis de ambiente

Crie o arquivo .env na raiz (NUNCA envie para o GitHub — adicione ao .gitignore):

```
DATABASE_URL=postgresql://postgres:senha123@db:5432/taskflow
SECRET_KEY=troque_por_uma_chave_aleatoria_de_32_chars
ALGORITHM=HS256
ACCESS_TOKEN_EXPIRE_MINUTES=30
POSTGRES_DB=taskflow
POSTGRES_USER=postgres
POSTGRES_PASSWORD=senha123
```

Atenção: no DATABASE_URL o host é 'db' (nome do serviço no Compose), não 'localhost'.

Variável	Descrição	Exemplo
DATABASE_URL	URL completa de conexão com o PostgreSQL	postgresql://postgres:senha@db:5432/taskflow
SECRET_KEY	Chave secreta para assinar os tokens JWT	string aleatória de 32+ caracteres
ALGORITHM	Algoritmo de assinatura JWT	HS256
ACCESS_TOKEN_EXPIRE_MINUTES	Minutos até o access token expirar	30

Variável	Descrição	Exemplo
POSTGRES_DB	Nome do banco criado automaticamente	taskflow
POSTGRES_USER	Usuário do PostgreSQL	postgres
POSTGRES_PASSWORD	Senha do PostgreSQL	nunca usar senha fraca em produção

2.4 .gitignore — o que NÃO versionar

```
.env
__pycache__/
*.pyc
.pytest_cache/
*.egg-info/
```

3. Containers do Projeto

Serviço	O que roda	Porta	Origem da imagem	Observação
api	Sua aplicação FastAPI	8000	Dockerfile local	Depende do db estar saudável
db	PostgreSQL 15	5432	postgres:15 (Hub)	Volume para persistir dados

4. Comandos Essenciais

Comando	O que faz
docker compose up	Sobe todos os containers (mostra logs no terminal)
docker compose up -d	Sobe em background (detached mode)
docker compose down	Para e remove os containers (dados do volume persistem)
docker compose down -v	Para, remove containers E apaga os volumes (perde dados do banco)
docker compose build	Rebuilda as imagens sem subir os containers
docker compose logs -f api	Acompanha os logs da API em tempo real
docker compose ps	Lista os containers rodando e seus status
docker compose exec api bash	Abre terminal dentro do container da API

4.1 Fluxo do dia a dia

1. Primeira vez: docker compose up --build (builda a imagem e sobe)
2. Demais vezes: docker compose up -d (sobe em background)
3. Ver logs se algo der errado: docker compose logs -f api
4. Parar tudo: docker compose down
5. Mudou o requirements.txt? Precisa rebuildar: docker compose up --build

5. Deploy — Colocando no Ar

Deploy é o processo de mover sua aplicação do ambiente local para um servidor na internet. Com Docker, o processo é muito mais simples porque o container garante que tudo funciona igual em qualquer lugar.

5.1 Comparativo de plataformas

Plataforma	Dificuldade	Custo	Observação
Railway	Muito fácil	Gratuito (limitado)	Conecta ao GitHub, detecta Dockerfile automaticamente. Melhor para começar.
Render	Muito fácil	Gratuito (limitado)	Similar ao Railway. Suporte nativo a FastAPI e PostgreSQL.
Fly.io	Médio	Gratuito (limitado)	Mais flexível, usa CLI. Bom para projetos maiores.
DigitalOcean	Médio	A partir de $6/mês	VPS com mais controle. Você gerencia o servidor.
AWS / GCP	Avançado	Variável	Mercado de trabalho. Complexo para começar.

5.2 Fluxo geral de deploy

Etapa	O que acontece	Detalhes
1. Código local	Você desenvolve e testa com docker compose up localmente	
2. Git push	Envia o código para o GitHub (sem o .env)	
3. CI/CD	Railway / Render detecta o push e inicia o build automaticamente	

Etapa	O que acontece	Detalhes
4. Build da imagem	A plataforma executa o Dockerfile e gera a imagem	
5. Deploy	Container sobe com as variáveis de ambiente configuradas na plataforma	
6. Migrações	Alembic roda automaticamente se configurado no CMD do Dockerfile	
7. Online	API disponível na URL pública gerada pela plataforma	

6. Deploy no Railway — Passo a Passo

Railway é a opção mais simples para iniciantes: detecta o Dockerfile automaticamente e provisiona o banco PostgreSQL com poucos cliques.

Passo	Ação	Detalhes
1	Criar conta	Acesse railway.app e crie uma conta com o GitHub
2	Novo projeto	Clique em 'New Project' → 'Deploy from GitHub repo'
3	Selecionar repo	Escolha o repositório do TaskFlow
4	Adicionar banco	Clique em '+ New' → 'Database' → 'PostgreSQL'
5	Variáveis de ambiente	Em 'Variables', adicione todas as variáveis do .env
6	DATABASE_URL	Railway injeta automaticamente — copie da aba do banco
7	Deploy automático	A cada push no GitHub, Railway faz redeploy automaticamente
8	Domínio	Em 'Settings' → 'Domains', gere um domínio gratuito .railway.app

6.1 Configurar variáveis no Railway

- No painel do projeto, clique no serviço 'api'
- Vá em 'Variables' → 'Raw Editor'
- Cole todas as variáveis do seu .env (exceto DATABASE_URL — o Railway injeta automaticamente)
- Clique em 'Update Variables' — isso dispara um novo deploy automaticamente

6.2 Verificar se está funcionando

- Acesse a URL gerada (ex: taskflow-api.railway.app/docs)
- O Swagger deve abrir normalmente
- Teste o endpoint GET /health — deve retornar {"status": "ok"}

7. Checklist de Segurança antes do Deploy

Item	Como garantir
Nunca versionar o .env	Adicionar .env no .gitignore antes do primeiro commit
SECRET_KEY forte em produção	Gerar com: python -c "import secrets; print(secrets.token_hex(32))"
CORS restrito em produção	Trocar allow_origins=['*'] pelo domínio real do frontend
Migrations antes de subir	Garantir que alembic upgrade head rode na inicialização
Health check configurado	Endpoint GET /health para a plataforma verificar se a API está viva
Logs sem dados sensíveis	Nunca logar senhas, tokens ou dados pessoais
Banco com senha forte	POSTGRES_PASSWORD com pelo menos 16 caracteres aleatórios

8. Problemas Comuns e Soluções

API não conecta no banco

- Verifique se DATABASE_URL usa 'db' como host (não 'localhost')
- Confirme que o healthcheck do PostgreSQL está passando: docker compose ps
- Veja os logs do banco: docker compose logs db

Porta 8000 já em uso

- Outro processo está usando a porta. Troque para 8001:8000 no compose temporariamente
- Ou encerre o processo: lsof -i :8000 (Mac/Linux) ou netstat -ano (Windows)

Mudanças no código não aparecem

- Docker usa cache da imagem anterior. Rebuild obrigatório: docker compose up --build

Dados do banco somem após docker compose down

- Você usou docker compose down -v que apaga volumes. Use apenas docker compose down

Deploy falha na plataforma

- Verifique os logs de build na plataforma (Railway/Render mostram o output do Dockerfile)
- Confirme que o requirements.txt está atualizado e commitado no GitHub
- Verifique se todas as variáveis de ambiente estão configuradas na plataforma

---

Fim do Documento — TaskFlow Docker & Deploy v1.0 | Abril 2026